Business 41204

Machine Learning

Policy Learning

Outline:

1. Introduction
2. Offline Policy Learning
3. Bandits
4. Contextual Bandits
5. Playing Games
6. More Involved Examples
7. Summary

1. Introduction

Policy Learning

Introduced policy learning in last set of notes

Supervised learning (SL), unsupervised learning (USL), traditional statistics:

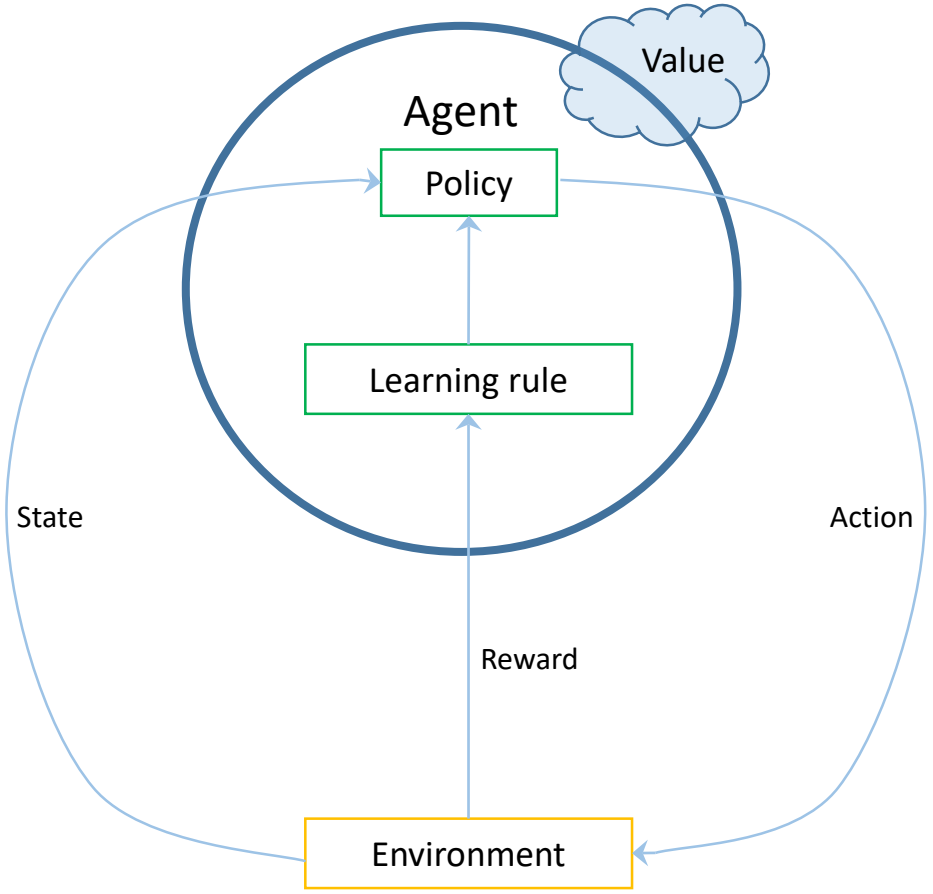
- Focus on extracting patterns that describe/summarize data

Policy learning:

- Focus on finding actions (or policies) that maximize rewards
  - Can use SL/USL to help estimate rewards
  - Model structures (e.g. trees, DNNs) can be used to parameterize policy and reward functions
- Explicitly targets data-driven/algorithmic decision making
  - Decision making is not a byproduct or afterthought
  - E.g. mean squared prediction error often tangentially related to decision making (at best)

Structure/Jargon

- **Agent:** learner/decision-maker trying to achieve some goal.
- **Environment:** context where agent operates
- **Action:** decision taken by agent.
- **State:** conditions under which agent takes actions at any given decision time. May change over time. May be influenced by actions. May have random/unknown elements.
- **Reward:** immediate feedback agent receives about action. May be positive or negative.
- **Policy:** rule that determines how agent behaves. Mapping between states and actions.
- **Value:** What we're really after. Basically the sum of rewards. Allows thinking about decisions that pay off at longer horizons.



A Simplified Setup

Start in setting where actions do not affect the environment

- Action today does not impact future rewards, states, …
- We'll relax this in reinforcement learning (later)

Goal: Find a *policy function*, $\pi(s)$, that tells us what action, a, to take when the state is s

Define the *value function for policy* π as

$$V(\pi) = E[R(\pi(S))]$$

- $R(a)$ is the (stochastic) reward received when taking action a

Optimal policy then solves $\max_{\pi} V(\pi)$

We'll look at solving this problem

- Based on historical data – offline policy learning
- When online experimentation/active learning is possible – multi-armed and contextual bandits

2. Offline Policy Learning

Experimental Data with Binary Treatment

Recall we already did policy learning in a setting with a completely randomized binary treatment

Chose $\pi(X) \in \{0,1\}$ to maximize

$$V(\pi) = E[\pi(X)CATE(X)] - E[\pi(X)Cost(X)]$$
$$= E\left[\pi(X)\tilde{Y}\right] - E[\pi(X)Cost(X)]$$

- $V(\pi)$ = value of policy π relative to baseline of "treating no one"
- $\tilde{Y} = \left(\frac{T}{p} - \frac{1-T}{1-p}\right)\big(Y - g(T,X)\big) + g(1,X) - g(0,X)$
  - NOTE: We're assuming an observed outcome is the reward we care about
- $Cost(X)$ = Cost of treating unit with characteristics X
- $g(T,X)$ = average reward given characteristics X in treatment state T
  - Estimated via favorite SL method
- T randomly assigned binary treatment with treatment probability p

Generalization

Suppose treatment T can be control (0) or one of K candidate treatments $\Rightarrow \pi(X) \in \{0,1,\dots,K\}$

- Could further generalize to continuous action space

Let $p_k(X)$ for $k = 0,1,\dots,K$ denote the probability of being assigned status k

- Treatment probability can depend on characteristics X
- Can be used with observational data under the assumption that there is no confounding after controlling for X
  - In this case, $p_k(X)$ can be estimated with favorite ML classification method

Relative to our binary example, we'll change the goal post a bit by not benchmarking to a baseline policy:

$$V(\pi) = E\left[Y\left(\pi(X)\right) - Cost(\pi(X), X)\right] = E\left[\tilde{Y}\left(\pi(X)\right) - Cost(\pi(X), X)\right]$$

Potential outcome under treatment status $\pi(X)$

for

- $\tilde{Y}(\pi(X)) = \left(\frac{1(T=\pi(X))}{p_T(X)}\right)\left(Y - g(T,X)\right) + g(\pi(X),X)$
- $Cost(\pi(X), X)$ = Cost of treatment $\pi(X)$ for unit with characteristics X
- $g(T,X)$ = average reward given characteristics X in treatment state T
  - Estimated via favorite SL method

Optimization Problem

Suppose we have

- n training observations
- $\hat{p}_k(X)$ – estimate of $\Pr(T = k|X)$ – for each $k = 0,1, \dots, K$
- $\hat{g}(T, X)$ – estimate of $E[Y|T, X]$
- 0 costs for simplicity

Would like to solve

$$\max_{\pi} \frac{1}{n} \sum_{i=1}^{n} \left(\frac{1(T_i = \pi(X_i))}{\hat{p}_{T_i}(X_i)} \right) \left(Y_i - \hat{g}(T_i, X_i) \right) + \hat{g}(\pi(X_i), X_i)$$

Assuming measured outcome corresponds to reward we care about. Often a real challenge.

- Challenging optimization problem because policy space is discrete
- Other ways to pose problem but none fundamentally resolves the issue
  - E.g. just directly maximizing reward by solving $\max_{\pi} \frac{1}{n} \sum_{i=1}^{n} \hat{g}(\pi(X_i), X_i)$ seems easier, but is still discrete

Practical Suggestions

1. Only consider (simple) parameterized policies
   - Policy rule: $\pi(X) = \pi(X, \theta) = \arg\max_k f_k(X, \theta)$ where f_k is a utility function
     - Linear score: $f_k(x, \theta) = x'\theta_k$
     - Tree-based rule: Form shallow tree based on value and assign fixed actions within leaves
       - Relatively easy to implement and interpretable
     - Neural net: $f_k(x, \theta)$ is class-specific probability from small neural network suitable for multi-class classification

2. Consider smoothing the policy.
   - Rather than assign deterministic policies, consider a policy that assigns probabilities to actions.
     - Call these probabilities $f_k(x, \theta)$
   - The expected objective under this (non-deterministic) policy is then

$$\frac{1}{n}\sum_{i=1}^{n}\sum_{k=0}^{K} f_k(X_i, \theta)\left[\left(\frac{1(T_i = k)}{\hat{p}_{T_i}(X_i)}\right)\left(Y_i - \hat{g}(T_i, X_i)\right) + \hat{g}(k, X_i)\right]$$

   - This objective is smooth in θ

Practical Suggestions

3. Remember your target – you are trying to learn a policy to use in the future
   - *At most*, traditional statistical uncertainty in the learned policy is secondary
     - Once it's learned, you're using the policy. The possibility that you could have learned something else with different data is irrelevant.
   - Have validation data set aside to benchmark how your fixed policy might have done on new data
     - Highly imperfect, especially with observational data, but still a good idea.

Code for Reemployment Bonus Example

Reemployment Bonus Example

Penn reemployment bonus experiment

- Experiment looking at policies aimed at reducing length of unemployment spells
- 5 treatment arms offering differing incentives for quickly finding full-time employment and job-search assistance
  - Some issues in deployment (e.g. started with six arms) which we'll ignore

Data:

- 13913 observations
- Outcome: number of weeks of unemployment
- 14 binary pretreatment covariates
- Dummies for time individual enters experiment

Goal: Find policy for assigning treatment arm based on pretreatment variables

Reemployment Bonus Example

Estimates of regression functions and treatment probabilities

- Estimated using only training data
  - Validation data held out for evaluating policies
- Treatment probabilities (multiclass classification)
  - Try three tree models, three random forests, and the unconditional frequency
  - Choose model based on five fold cross validation
    - Selected model: Depth 2 tree
- Regression functions
  - Fit separately on each treatment arm
  - Try linear regression, three tree models, three random forests
  - Choose model based on five fold cross validation
    - Selected model: Linear regression

Reemployment Bonus Example

Solve policy optimization problem

1. Raw problem with tree of depth 2
2. Smooth problem with probabilities linear index inside logistic link
   - Make deterministic by then assigning treatment with highest “probability”
- In both cases, policy assignment will depend on individual pretreatment characteristics

Compare to simple policies that just assign same treatment to all observations

Evaluate in validation data

Reemployment Bonus: 0 costs

Let's start by pretending costs are 0.

Estimated value (average weeks unemployed)

	Training Data		Validation Data	
	DR	s.e.	DR	s.e.
Control	13.29	0.23	13.37	0.29
Treatment 1	13.06	0.37	12.37	0.46
Treatment 2	12.51	0.26	13.03	0.33
Treatment 3	13.23	0.31	12.70	0.39
Treatment 4	12.31	0.26	12.94	0.35
Treatment 5	12.79	0.32	13.62	0.40
Tree policy	11.81	0.30	12.63	0.38
Smoothed policy	10.94	0.27	12.58	0.35

Assignments:

	Tree	Smooth
Control	0	0
1	552	85
2	0	415
3	1373	205
4	2552	4405
5	1089	456

Reemployment Bonus: Non-zero costs

For fun, let's suppose costs are (0,.5,1,1,1.5,2) (in terms of weeks of unemployment)

Estimated value (average weeks unemployed)

	Training Data		Validation Data		
	DR	s.e.	DR	s.e.	Ave Cost
Control	13.29	0.23	13.37	0.29	0
Treatment 1	13.56	0.37	12.87	0.46	.5
Treatment 2	13.51	0.26	14.03	0.33	1
Treatment 3	14.23	0.31	13.70	0.39	1
Treatment 4	13.81	0.26	14.44	0.35	1.5
Treatment 5	14.79	0.32	15.62	0.40	2
Tree policy	12.82	0.26	13.14	0.34	0.15
Smoothed policy	11.91	0.27	13.28	0.36	0.35

Assignments:

	Tree	Smooth
Control	4215	3022
1	1069	1501
2	282	912
3	0	0
4	0	0
5	0	131

3. Bandits

Multi-armed bandits

Problem:

- n possible actions with unknown payoffs
- Need to repeatedly choose among these actions
  - Each time you choose, you get a random payoff depending on the action taken
  - Distribution generating rewards is not influenced by action
- Goal is to maximize (expected) total reward after some number of actions
- Solution is trivial with known payoff (distributions)
  - Always choose the one with highest (expected) payoff!
- Actions don't influence states and no worry about short term actions impacting long term rewards

Examples:

- Have several potential advertisements you could run and want to choose the one that leads to the largest increase in purchases/revenue
- Have several candidate (experimental) treatments for a condition and want to choose the one that leads to the largest increase in well-being of patients

Exploit vs Explore

"Traditional Statistics" solution:

- run an experiment with many observations per potential treatment
  - Simple, robust. "Gold standard" for scientific discovery
  - BUT, inefficient – wastes resources learning about options that don't pay off
  - Focuses on *exploration*

"Greedy" solution:

- try each option a small number of times and then just stick with one that gave highest reward
  - Potentially beneficial in getting actual payoffs as "bad" policies are discarded
  - Great in a world with "stable" outcomes
  - BUT, prone to mistakes – does not spend resources *learning* to distinguish between different outcomes when the policy does not deliver "stable" outcomes
  - Focuses on *exploitation*

"Optimal" solution trades off exploration and exploitation

- Want to actually learn rewards of different choices without spending lots of time making bad decisions
- Under structure (likely unrealistic), can determine sophisticated strategies to produce theoretically best tradeoff between exploration/exploitation
  - Not going to worry about this, but give ideas and talk about things that seem to work pretty well in practice

ε-Greedy Algorithms

Basic setup:

- At time *t+1*, know $\bar{X}_{j,t} = \frac{1}{N_{j,t}} \sum_{\tau=1}^{t} 1(arm\ j\ used\ at\ time\ \tau) x_{j,\tau}$ for $j = 1, \dots, n$
  - $x_{j,\tau}$ = payoff from trying action j at time τ
  - $N_{j,t} = \sum_{\tau=1}^{t} 1(arm\ j\ used\ at\ time\ \tau)$ = number of times action j tried up to time t
  - I.e. know average payoff up to time t
- Need to choose action for time $t + 1$

ε-Greedy Algorithm:

- Choose action with highest $\bar{X}_{j,t}$ with probability 1-ε
- Choose random action with probability ε

Idea:

- Forces all options to eventually be explored many times
- Spends a lot of time on what seems to be the best action
  - Can get stuck spending a lot time on “ok” actions

Upper Confidence Bound (UCB) Algorithm

Basic setup:

At time *t+1*, know $\bar{X}_{j,t} = \frac{1}{N_{j,t}} \sum_{\tau=1}^{t} 1(arm\ j\ used\ at\ time\ \tau) x_{j,\tau}$ for $j = 1, \dots, n$

- $x_{j,\tau}$ = payoff from trying action j at time τ
- $N_{j,t} = \sum_{\tau=1}^{t} 1(arm\ j\ used\ at\ time\ \tau)$ = number of times action j tried up to time t
- I.e. know average payoff up to time t

Need to choose action for time $t + 1$

Upper Confidence Bound (UCB) Algorithm

UCB Algorithm:

- Choose action with highest UCB Score:

$$UCB_j(t) = \bar{X}_{j,t} + \sqrt{\frac{2\ln(t)}{N_{j,t}}}$$

  - Can be adapted to incorporate the sample variance as well

Idea:

- Optimistic and deterministic search algorithm
  - Prioritizes high reward $\bar{X}_{j,t}$
  - Encourages more exploration for options that have not been taken much (small $N_{j,t}$)
  - $\ln(t)$ terms encourages lots of exploration initially that slows over time (but because $\ln(t)$ is increasing and unbounded, always some "pressure" to explore)
    - $\ln(t)$ specifically because of concentration results (Hoeffding's inequality)

Code for Bandit Example

Bandit Illustration

Consider a stylized bandit example.

Need to choose among 10 options over a horizon of 1000 periods.

- At each period, reward for action j drawn from $N(\mu_j, 1)$

Look at the performance of 5 algorithms ("agents")

- Completely random
- ε-greedy with $\varepsilon = 0.1$
- Completely greedy
- UCB
- Thompson Sampling
  - A probabilistic algorithm based on Bayesian updating

Bandit Illustration

Looks like completely greedy is doing really well!

Do we think this generalizes beyond this one example?

Performance in one simulation:

	Completely Random	Epsilon-Greedy	Completely Greedy	UCB	Thompson Sampling
Average Reward	-0.13	1.54	1.76	1.66	1.69
Optimal Actions	108	892	988	913	938

Action 8 is the optimal action.

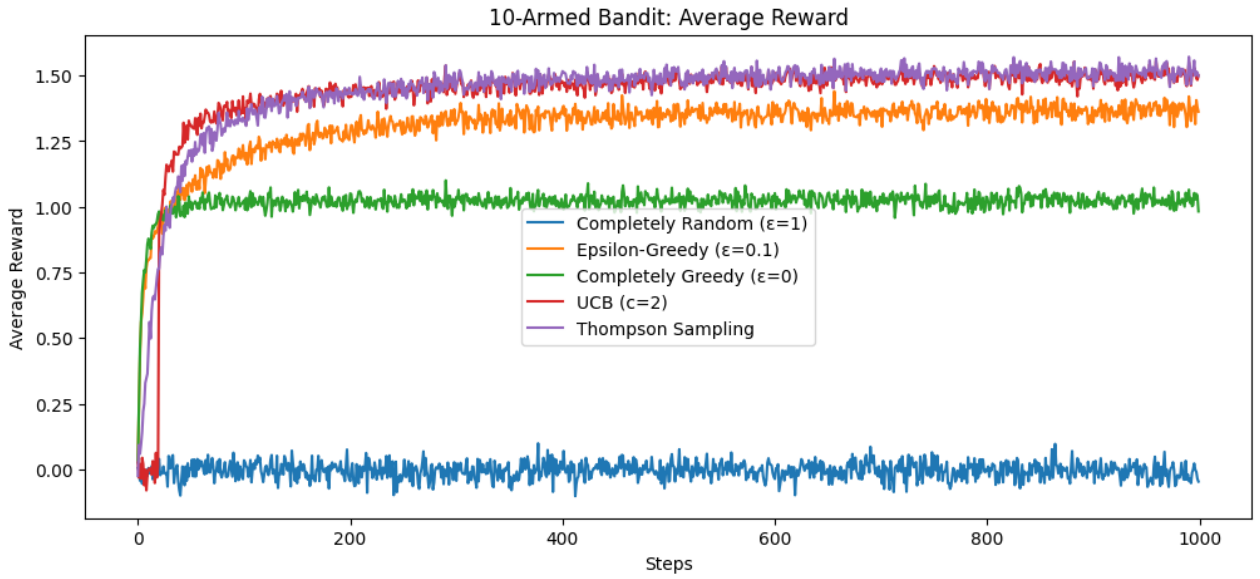
All algorithms learn the payoff to Action 8 pretty well.

All algorithms learn Action 8 is much better than other actions.

What if we are interested in payoffs to other actions?

	True Reward	Completely Random	Epsilon-Greedy	Completely Greedy	UCB	Thompson Sampling
Action 1	0.23	0.18	0.40	-0.69	0.17	0.17
Action 2	1.03	1.18	0.89	-0.25	0.99	1.15
Action 3	-0.84	-0.93	-0.09	-0.44	-2.05	-0.86
Action 4	-0.59	-0.72	-0.60	-0.73	-1.56	-0.75
Action 5	-0.96	-1.00	-0.86	-0.44	-0.90	-0.68
Action 6	-0.22	-0.21	-0.46	-0.02	-0.87	0.14
Action 7	-0.62	-0.60	-0.89	-1.66	-0.57	-0.44
Action 8	1.84	1.93	1.77	1.78	1.78	1.78
Action 9	-2.05	-1.95	-2.20	0.00	-1.98	-1.13
Action 10	0.87	0.72	0.59	0.00	0.71	0.82

Bandit Illustration



Performance across 2000 simulations

- Each simulation uses different mean rewards for the actions

Figures show

- average (across 2000 simulations) reward at each time step
- fraction of instances (across 2000) where optimal action is selected at each time step

Algorithm	Average MSE (Optimal Action)	Average MSE (Across All Actions)
Completely Random	0.011	0.010
ε-Greedy	0.022	0.085
Completely Greedy	1.896	0.852
UCB	0.047	0.271
Thompson Sampling	0.009	0.130

Comments on Bandits

Things to think about:

- What would you do if you think the reward (distribution) changes over time?
- What do you think happens as you add more possible actions?
- What if action payoffs are similar?
  - What if action payoffs are highly variable?

- What is your goal?
  - Learn about all action payoffs as well as possible?
  - Choose the best action as much as possible?

4. Contextual Bandits

Contextual bandits

Contextual bandits solve same problem as bandits

BUT, now want to use "context"

- i.e. characteristics observed at time of action choice

Idea: Different actions may have different payoffs for units with different characteristics

- Movie recommendations
  - Context: viewing history
  - Customers with different viewing histories probably respond differently to identical recommendations
- Airline pricing/offers
  - Context: origin and destination, how far away is trip, length of stay, season of trip, travel history, …
  - Same fare/offer may perform differently for different travelers

Adapt ideas from supervised learning to model rewards as function of context

Linear UCB

In basic UCB, each action is given a single unknown average payoff

Linear UCB adapts to have payoffs depend *linearly* on *observed context*

- Context vector: w_t
- For each action $j = 1, \dots, n$, assume expected payoff satisfies

$$E\left[x_{j,t} \middle| w_t\right] = \beta_{j0} + \beta_{j1} w_{t,1} + \cdots + \beta_{jK} w_{t,K}$$

- Coefficients $\beta_{j0}, \dots, \beta_{jK}$ for $j = 1, \dots, n$ unknown – learned from data
- Goal is now to learn *payoff function* (not just payoff average)
- Context can help predict payoffs in new contexts (under assumed payoff structure)

To choose action at time t, observe $\left\{\left(w_\tau, x_{j(\tau),\tau}\right)\right\}_{\tau \le t}$

- I.e. observe context and payoff for action taken at every preceding time period
- Use to estimate coefficients with (regularized – e.g., lasso, ridge) linear regression

Linear UCB Algorithm Sketch

1. At time t, compute
   - Predicted payoff: $\hat{\mu}_j(w_t) = \hat{\beta}_{j0} + \hat{\beta}_{j1} w_{t,1} + \cdots + \hat{\beta}_{jK} w_{t,K}$
   - Uncertainty (akin to standard error from UCB): $\hat{\vartheta}_j(w_t)$
   - UCB score: $\mathrm{UCB_j(t)} = \hat{\mu}_j(w_t) + \alpha \hat{\vartheta}_j(w_t)$

   for each action j

2. Choose action with highest UCB score
   - Can update parameters for use at next instance after observing outcome

Linear UCB

Intuition same as basic UCB:

- Exploit actions with high predicted payoffs
- Explore actions with high uncertainty
- Payoffs and uncertainty now depend on which context we observe
  - Uncertainty will be higher for new/rare user types, situations

Key ideas:

- Optimistic, deterministic policy (like UCB)
- Generalizes UCB by replacing sample means with regressions
- Exploration happens where the model is least certain

Linear regression can be replaced by other structures

- E.g., trees – handle interactions automatically, clear visual structure
- E.g., DNNs – *deep contextual bandits*
- UCB idea remains the same

Code for Movie Example

Movie Example

Look at contextual bandit for recommending movies

- movie genre specifically

- Data: MovieLens ratings data
  - Users rate movies on 1-5 scale
  - Focus on a subset of popular movies

- Actions: Choose movie genre (e.g. action, comedy) to recommend

- Context: User's past viewing history represented as shares of movies watched in each genre

- Reward: 1 if user views movie of recommended genre *and* rates the movie highly. 0 otherwise
  - We're using historical data to *simulate* online bandit learning. We're really asking if the recommended genre would have been a good choice.

Goal: Learn which genres work for users with different viewing patterns

Movie Example

We have 19 possible actions:

- Action, Adventure, Animation, Children, Comedy, Crime, Documentary, Drama, Fantasy, Film-Noir, Horror, IMAX, Musical, Mystery, Romance, Sci-Fi, Thriller, War, Western
- Note that a movie will typically belong to several genres

Context is a user profile constructed from past viewing history

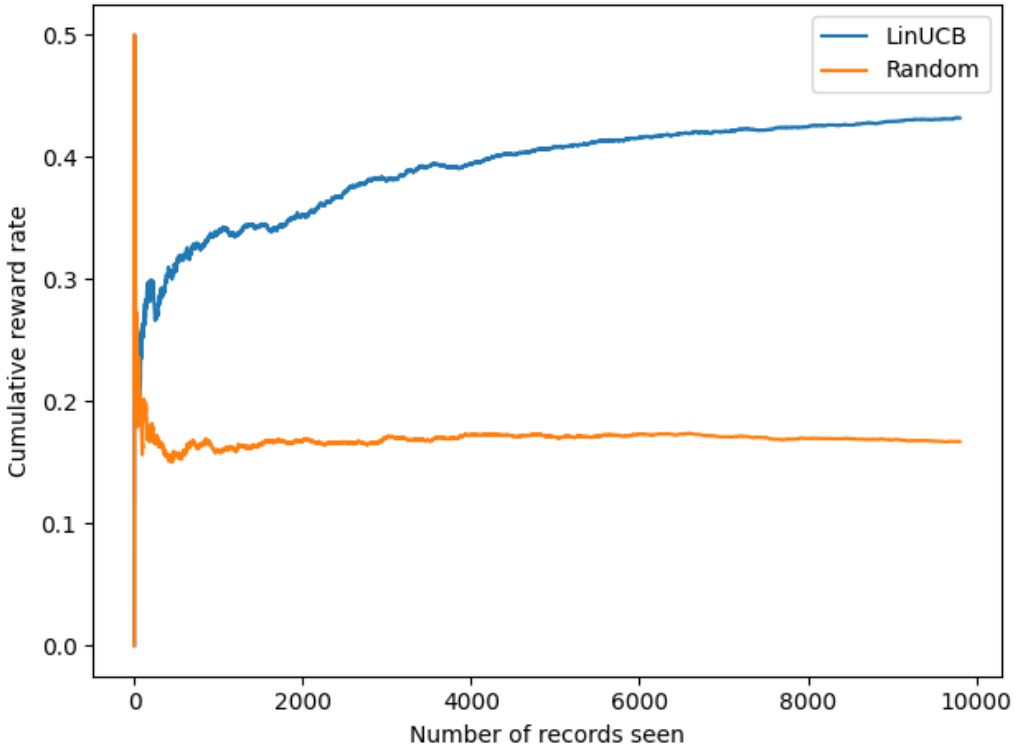
- E.g. user 610's previous viewing includes 14% action movies, 7% adventure, …

userId	Action	Adventure	Animation	Children	Comedy	Crime	…
610	0.14	0.07	0.02	0.02	0.11	0.07	…

Bandit uses this context to predict genre likely to perform well for each user as they arrive

Movie Example: Bandit Performance

Contextual bandit performance relative to random recommendation:



Cumulative reward: Fraction of recommendations that resulted in a high user rating up to that point

Bandit clearly outperforms random recommendations

Learns relatively quickly (in this example)

Movie Example: Interpretation

Recommendation Fractions

	Bandit	Random
Action	0.5875	0.0495
Drama	0.1683	0.0513
Adventure	0.0928	0.0502
Thriller	0.0907	0.0501
Comedy	0.0392	0.0549
Animation	0.0080	0.0562
Fantasy	0.0053	0.0555
IMAX	0.0034	0.0572
Mystery	0.0021	0.0493
War	0.0004	0.0548
Horror	0.0004	0.0540
Sci-Fi	0.0003	0.0541
Romance	0.0003	0.0529
Western	0.0003	0.0545
Film-Noir	0.0003	0.0484
Children	0.0002	0.0490
Documentary	0.0002	0.0520
Musical	0.0002	0.0509
Crime	0.0001	0.0554

Average Reward by Recommended Action

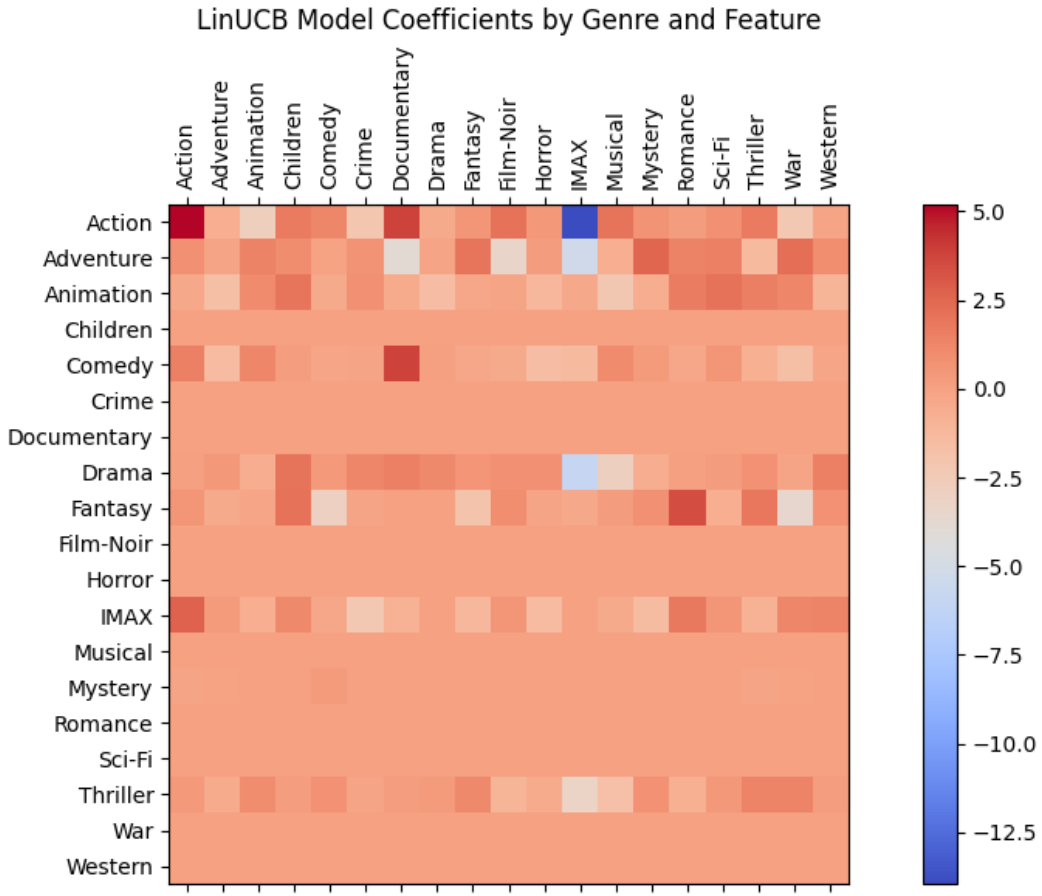
	Average Reward
Action	0.4707
Adventure	0.4077
Drama	0.4070
Thriller	0.3813
Comedy	0.3177
IMAX	0.1212
Animation	0.1154
Fantasy	0.1154
Mystery	0.0952
Documentary	0.0000
Children	0.0000
Horror	0.0000
Film-Noir	0.0000
Crime	0.0000
Musical	0.0000
Romance	0.0000
Sci-Fi	0.0000
War	0.0000
Western	0.0000

Have learned that certain genres associated with higher rewards.

Consistently make these recommendations.

Good thing?

Movie Example: Interpretation



Looking at coefficients in reward function for each arm

Row gives arm

Column gives context feature

5. Playing Games

Introduction to Reinforcement Learning

The Reinforcement Learning Problem

RL aims to find a *policy function*, $\pi(s)$, that tells us what action, a, to take when the state is s

- Difference relative to previous cases is that actions may impact environment

Define the *state value function for policy* π as

$$v_\pi(s) = E_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,|S_t = s\right]$$

- R_t is the reward at time t and γ is a discount factor
- Expected reward for following policy π when the state is s

Define the *action value function for policy* π as

$$q_\pi(s,a) = E_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,|S_t = s, A_t = a\right]$$

- Expected reward under policy π for taking action a when the state is s

The Reinforcement Learning Problem

Optimal policy then solves $\max_{\pi} v_{\pi}(s)$ AND $\max_{\pi} q_{\pi}(s,a)$

- Recall $v_{\pi}(s)$ denotes expected reward from following policy π when current state is s
- Recall $q_{\pi}(s,a)$ denotes expected reward from taking action a when current state is s and then following policy π

- Just trying to find policy that maximizes total/long term reward
- Let $v^*(s) = \max_{\pi} v_{\pi}(s)$ AND $q^*(s,a) = \max_{\pi} q_{\pi}(s,a)$

- Note that $\max_a q^*(s,a) = v^*(s)$

Bellman Equations

Unpacking things more:

$$
\begin{aligned}
(i)\ q^*(s,a) &= \max_{\pi} E_{\pi}\left[R_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k R_{t+k+2}\,|S_t = s, A_t = a\right] \\
&= E[R_{t+1} + \gamma v^*(S_{t+1})|S_t = s, A_t = a] \\
&= E[R_{t+1} + \gamma \max_{a'} q^*(S_{t+1}, a')|S_t = s, A_t = a]
\end{aligned}
$$

- The expected payoff from taking action a in state s now and then following the optimal policy is just the expected payoff from that decision today plus the expected payoff from following the optimal policy after that

Bellman Equations

Unpacking things more:

$$(ii)\ v^*(s) = \max_a q^*(s,a)$$
$$= \max_a E[R_{t+1} + \gamma v^*(S_{t+1}) | S_t = s, A_t = a]$$

- The expected payoff of following the optimal policy at state s must equal the expected payoff of taking the best action from that state

(i) and (ii) are Bellman equations

- Solving or approximating these equations underlie most RL algorithms
- Chief intuition just that we don't think ONLY about rewards now but also expected payoffs from actions in the future

Infinite Deck Blackjack

Let's look at a simple RL example. We want to learn how to optimally play infinite deck blackjack

- Infinite decks means card counting is irrelevant
- Our goal is playing – i.e. trying to win – we are not going to worry about betting
  - Really simplifies our problem!

Problem Formulation:

- Agent: Player
- Actions: hit/stand (ignore double down, split, surrender)
- Environment: deck of cards, dealer's rules
- State variables: sum of player's cards (12-21: why?), whether player has a useable ace, dealer's card (1-10)
  - 200 total relevant states
- Rewards: Win (+1), Draw (0), Loss (-1)
  - Only receive reward when a game (*episode*) ends

Solving with Q-Learning

There are many algorithms for solving RL problems. Here, we'll present a simple one called Q-learning:

- Basic idea is trying to learn the *optimal* Q-value function:

$$Q(s,a) = E[R_t | S_t = s, A_t = a]$$

  - In words, the expected reward at time t when the state of the world is s for taking action a
  - If we had the *optimal* Q-value function (call it $Q^*(s,a)$), we'd know what to do in state s: Choose the action that maximizes $Q^*(s,a)$

Solving with Q-Learning

Q-learning "Algorithm"

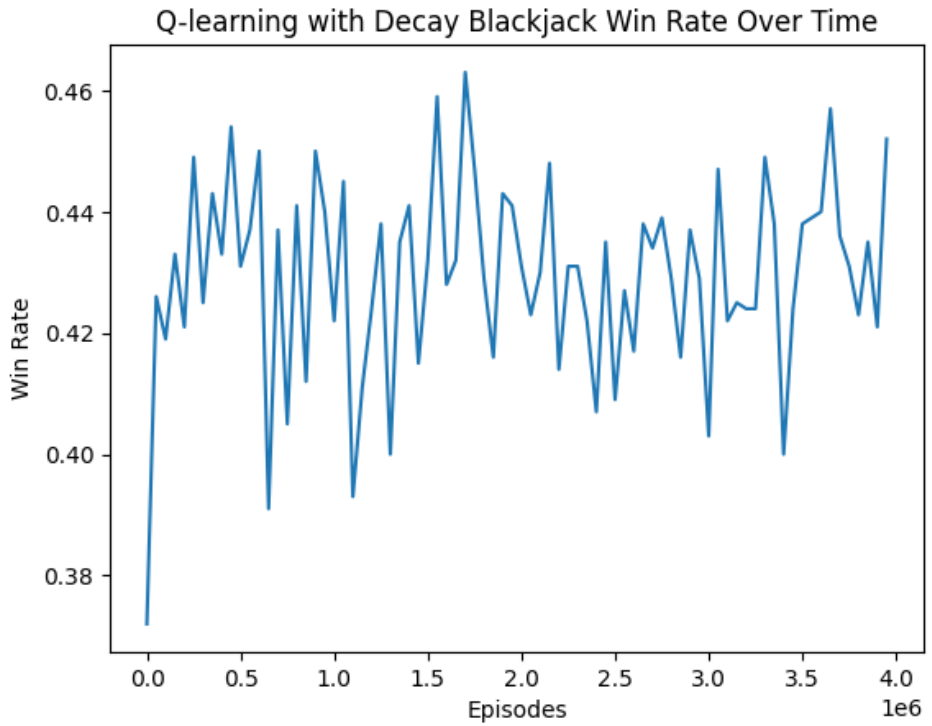
1. Make an initial guess for $Q(s, a)$
2. Repeat ***many*** times:
   1. Start a game (i.e. deal cards to player and dealer) and observe the initial state
   2. Use ε-greedy action choice:
      - Choose a random action with probability ε
      - Choose action $\arg\max_a \ Q_{cur}(s, a)$ where $Q_{cur}(s, a)$ is the current estimate of the Q-value function
   3. Perform action a, receive reward r, observe new state s'
   4. Update the Q-value function: $Q_{new}(s, a) = Q_{cur}(s, a) + \alpha \left[r + \gamma \max_{a'} Q_{cur}(s', a') - Q_{cur}(s, a) \right]$
      - α ("learning rate") and γ ("discount factor") are tuning parameters that control how quickly we update the Q function and how much we discount future rewards
      - Note this is essentially updating by approximating the Bellman equation
   5. Keep going until episode (game) ends and we observe win/loss/draw

Code for Blackjack Example

Blackjack Solution

Here, we run Q-learning for 4,000,000 episodes

- In this example, an episode is a hand of blackjack



Win rate of agent as it plays more hands.

Looks like it has learned to win 42-46% of hands (give or take)

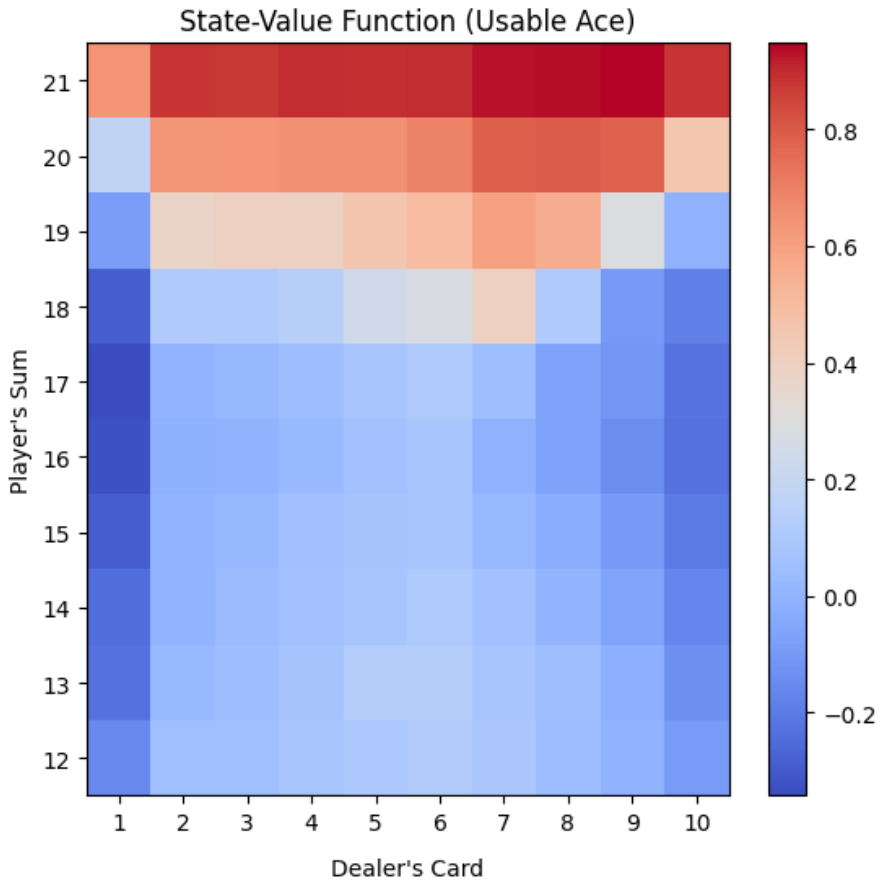
Humans win around 42% (https://www.winstar.com/blog/blackjack-odds/)

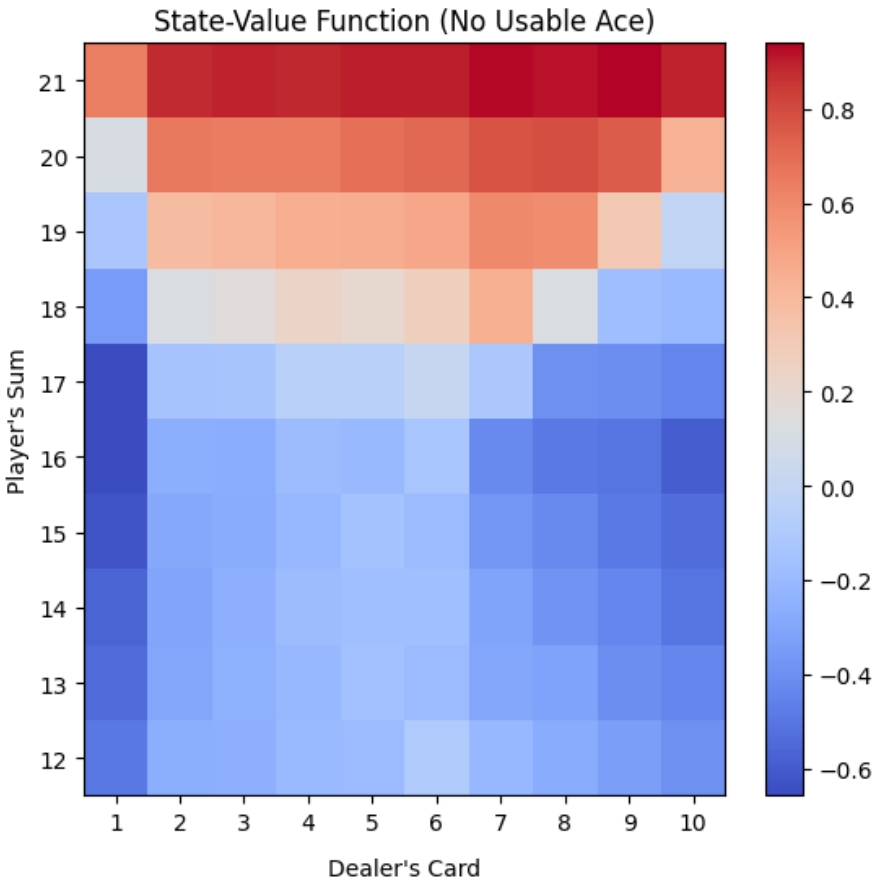
Blackjack State Value Function

Recall that we have defined our rewards as

- +1 (win)
- 0 (draw)
- -1 (lose)

The state value function gives you the expected reward from taking the optimal action in each state.



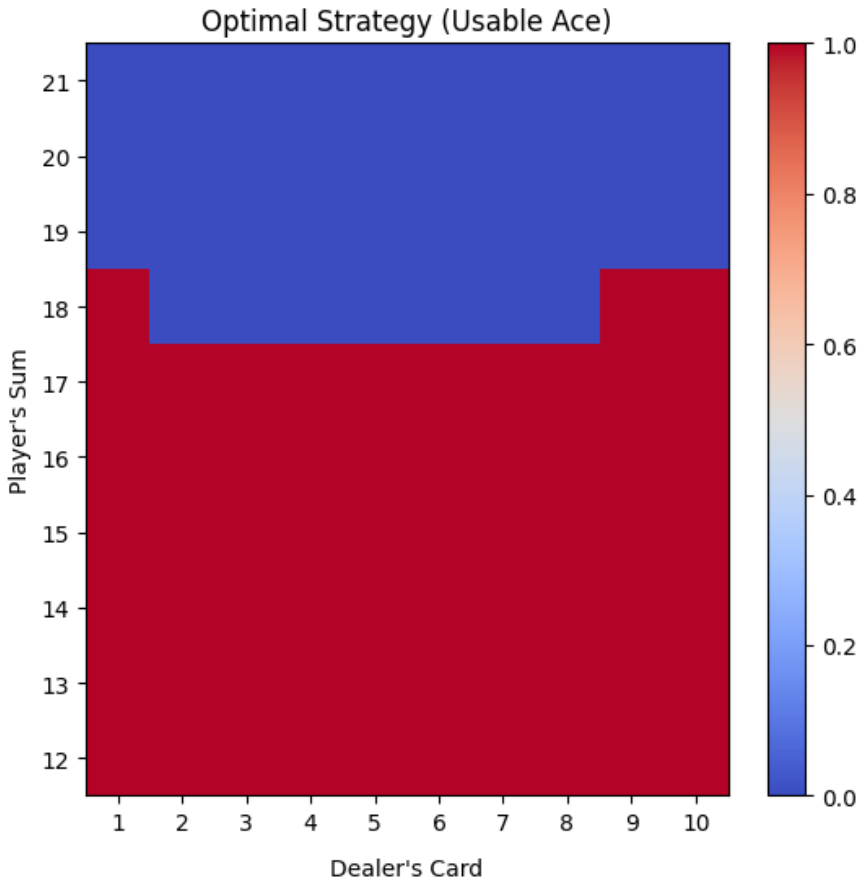


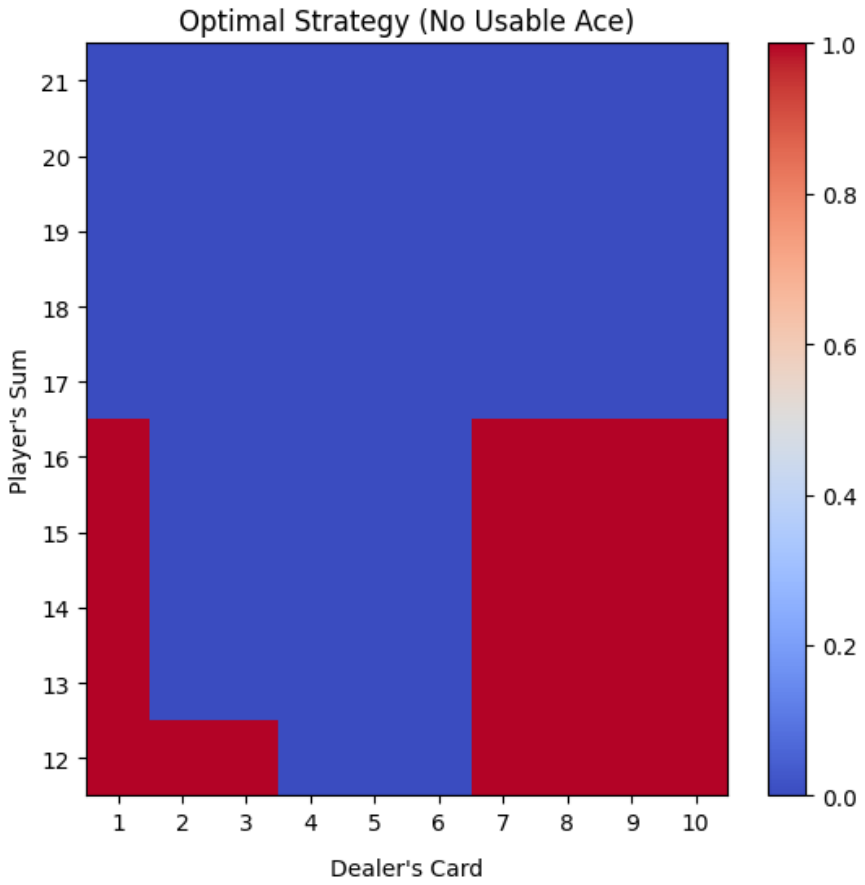
Blackjack Optimal Actions

Red denotes *hit*

Blue denotes *hold*

Almost identical to the “basic” strategy in Thorp (1966) Beat the Dealer





6. More Involved Examples

Large State Spaces

Let's think about single deck blackjack.

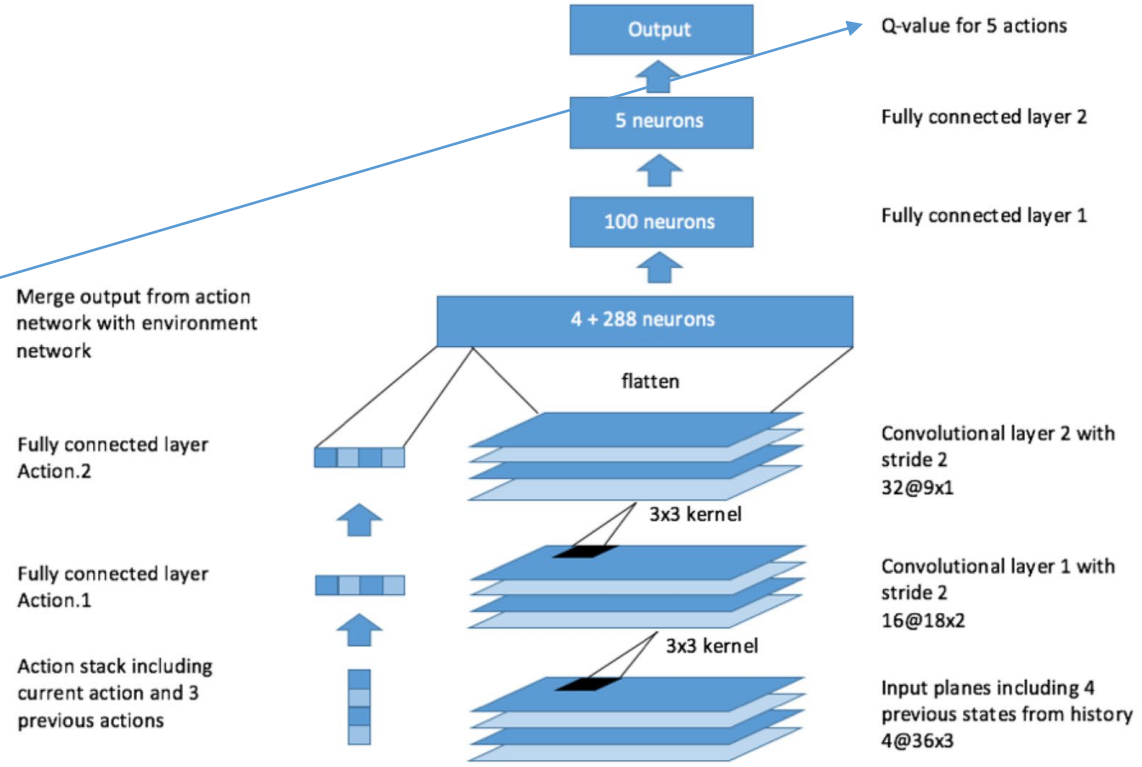
- With a single deck, card counting becomes relevant
  - To keep track of cards, need to enlarge the state space
  - Could imagine keeping track of all the cards left in the deck
  - State space is gigantic ≈ 840 million states
    - Could consider simpler counting strategies, but state space is still large
- Impractical to keep track of all states and hope to visit them many times!

RL with large input space proceeds by parameterizing action, value, and/or q-value functions

- E.g. specify them as linear models
- E.g. DeepRL – specify unknown functions as deep neural networks

Implementation typically requires

- Lots of computation
- Trial and error and tuning
  - Or luck ☺



AlphaGo

AlphaGo – AI trained to play Go

Go = Game like chess with many more possible moves

- Go: 19x19 board ($\approx 10^{170}$ states) vs Chess: 8x8 board ($\approx 10^{47}$ states)
- Go: ≈ 200-300 possible moves/turn vs Chess: ≈ 35 possible moves/turn
- Go: ≈ 200-300 moves/game vs Chess: ≈ 40-60 moves/game

Made waves in 2016 by soundly beating top human players

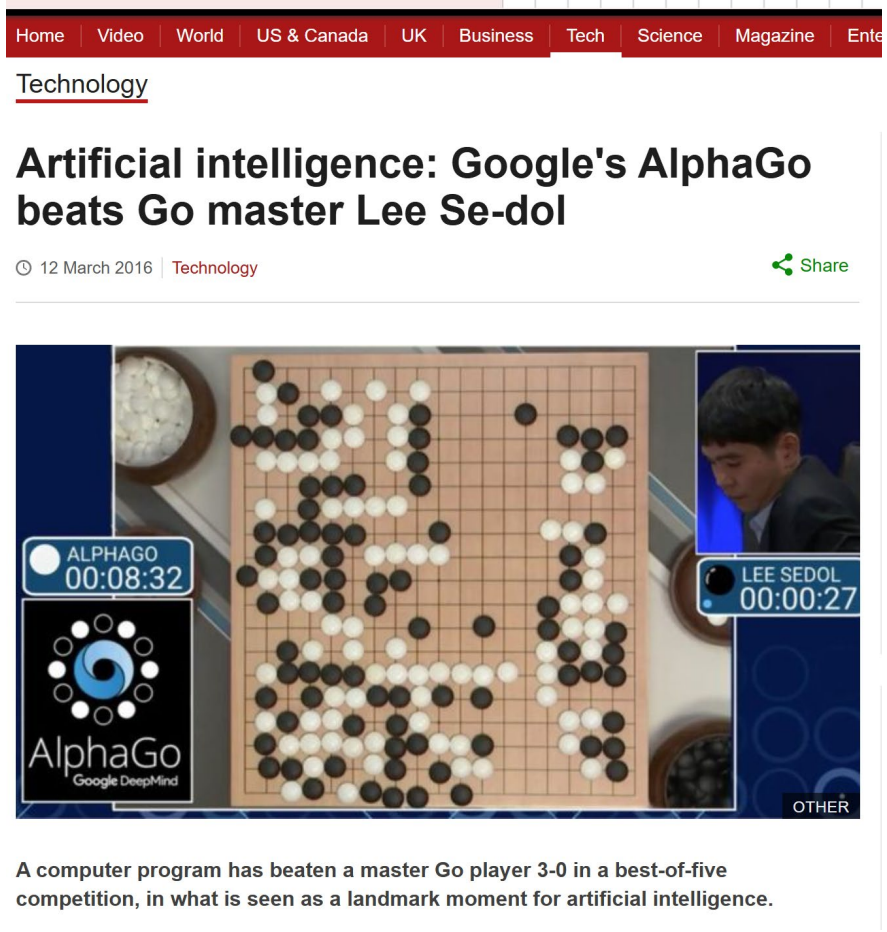
- Similar to chess breakthrough in 1997 but thought to be much harder

"We created AlphaGo, an AI system that combines deep neural networks with advanced search algorithms.

One neural network — known as the 'policy network' — selects the next move to play. The other neural network — the 'value network' — predicts the winner of the game.

Initially, we introduced AlphaGo to numerous amateur games of Go so the system could learn how humans play the game. Then we instructed AlphaGo to play against different versions of itself thousands of times, each time learning from its mistakes — a method known as reinforcement learning. Over time, AlphaGo improved and became a better player."
https://deepmind.google/research/breakthroughs/alphago/ (accessed 2/14/25)

"Artificial intelligence: Google's AlphaGo beats Go master Lee Se-dol". BBC News. 12 March 2016. Archived



AlphaGo

AlphaGo project formed around 2014

Estimated training cost (from https://klu.ai/glossary/alphago):

- $35M energy
- $25M hardware (trained on Google TPUs – specialized processors)
- ?? Human capital
- Not sure I buy these numbers and would certainly be much cheaper now
  - Hardware is cheaper and better
  - Algorithms are better
  - AI engineers have better understanding
    - AlphaGo played millions of games and trained for months, AlphaGo Zero (2017) played 4.9 million games and trained for 40 days, AlphaZero (2017) played 21 million games and trained for 3 days
    - Came across estimates of hundreds of thousands of dollars and several days using more current tech (still no human capital)

Building big RL systems can be expensive

Dynamic Pricing

Fixed resource (e.g. hotel rooms, airline seats), time-varying customer demand, ability to pre-book

- Agent: pricing algorithm
- Actions: set of allowed prices/price adjustments
  - What might one think about here?
- Environment: marketplace
- State space: market conditions, customer type → Can be contentious!
  - current price, competitor prices, time of day, seasonality, demand conditions, supply conditions, business/leisure, …
- Reward
  - Revenue?
  - Customer satisfaction?
  - What can be measured?

Implementation in the real world

- How long to accumulate enough data?
- What happens during training period? Monitoring and guardrails?
- “Stationarity”?
- Noise?

7. Summary

Summary

Reinforcement learning is computerized learning by doing

Bandits leading example that is easy to understand and use

RL has lots of potential applications

- Robotics, self-driving cars, …
  - Great (though complex) applications because can use simulated environments = lots of data
- Be wary of overclaiming simplicity/scalability though
  - RL for medicine?